

Trade Market Manipulation Detection with Graph Neural Networks: A Calibrated-Synthesis-to-Deployment Surveillance Pipeline

Sumit Sontakke

Department of Computer Science, PES University, Bengaluru, India
`pes2pge24ds1110stu.pes.edu`

June 6, 2026

Abstract

We describe an end-to-end surveillance pipeline for detecting collusive trade-market manipulation that ties together five components: a daily NSE bhavcopy calibration loop, a Cont-2001-stylised synthetic market generator, an agent-based simulator (ABIDES) used as a second substrate to test generator-shift, a graph neural network detector with an optional second-stage classifier, and a Streamlit investigation dashboard that surfaces flagged traders with per-feature evidence and a language-model justification card. We argue that the pipeline is the contribution: each layer was driven by a specific surveillance requirement (interpretability for investigators, robustness across parameter shifts, controllable false-positive cost), and the layers interlock through a small set of explicit interfaces. We instantiate the detector layer with three architectures—a 2-feature GraphSAGE baseline, a gradient-boosted bolt-on classifier consuming the GraphSAGE trader score plus six engineered features, and a feature-augmented GraphSAGE that ingests the same six features as node attributes—and report results across within-generator, cross-generator, and leave-one-family-out regimes. The augmented network lifts within-OOD edge AUC from 0.638 to 0.984; the bolt-on classifier reaches pooled trader AUC 0.968; the augmented network retains AUC 0.842 across a different simulator while the baseline collapses to chance. We discuss the practical implication: per-generator threshold calibration, an investigator dashboard, and an interpretable feature representation are at least as important as model complexity.

Keywords: Market surveillance, collusive trading detection, graph neural networks, calibrated synthesis, agent-based simulation, investigation dashboard, deployment, operating-point selection.

1 Introduction

Trade-market surveillance systems sit at the intersection of three distinct engineering pressures. They must satisfy a regulatory consumer that wants explicit, defensible rules; they must satisfy an investigator team whose workload determines the operationally acceptable purity floor; and they must satisfy a machine-learning team that wants to exploit modern representational learning to lift recall beyond what hand-crafted rules can reach. Each of these constituents pulls in a different direction. The investigator team wants per-flag evidence and a clean operating point at high precision. The regulator wants the rule or model to be auditable and stable. The ML team wants the freedom to use whatever architecture lifts AUC the most.

This paper does not present a single best model. Instead it presents a pipeline—five interlocking components—designed so that the three pressures co-exist. The pipeline begins with daily NSE bhavcopy ingestion and calibration of a synthetic market generator to the empirical first moments of the index. The generator is then driven with three manipulation taxonomies (clique, ring, mixed) at controllable parameter envelopes to produce labelled training cohorts that are both realistic enough to transfer and varied enough to expose distribution shift. A second substrate (the ABIDES agent-based simulator) is used in parallel as a cross-generator stress test. A graph neural network backbone, optionally augmented with a second-stage classifier, sits at the centre of the detection layer; six explicitly engineered per-trader features provide both the lift and the interpretability hooks the investigator team needs. A Streamlit dashboard surfaces the flagged traders, the price/volume context of the manipulated ticker, the counterparty network, and a per-trader language-model justification.

We claim three contributions. First, we describe

the pipeline as a whole, including the data interfaces and the small set of design decisions that connect each layer. Second, we report empirical results across three increasingly demanding cohorts, framed as system tests: within-generator out-of-distribution, cross-generator (a different simulator entirely), and a leave-one-family-out variant in which one manipulation taxonomy is held out from training. Third, we describe the operating-point and threshold-calibration logic that we believe is the correct deployment hygiene for a model whose ranking is stable across generators but whose absolute score distribution is not.

The paper proceeds as follows. Section 2 situates the work against existing market-surveillance and graph-fraud literature. Section 3 describes the pipeline at the system level and lists the interfaces between components. Section 4 describes the data pipeline (calibration, synthesis, ABIDES integration). Section 5 describes the six engineered features as surveillance signals. Section 6 describes the three detector variants we compare. Section 7 discusses the trader-projection and operating-point logic. Section 8 sketches the investigator dashboard. Section 9 reports empirical validation across the three test cohorts. Section 10 discusses deployment considerations. Section 12 covers limitations and Section 15 concludes.

2 Related Work

The literature on market-manipulation detection has historically been dominated by rule-driven systems that emit alerts when explicit microstructure conditions hold: layering thresholds on order cancellation rates, spoofing tests on persistence of unfilled top-book orders, pump-and-dump signatures on price-volume spikes. Khanna and Aggarwal [1] provide a useful survey of the rule era. The shift to supervised classifiers came later, motivated by the observation that rule-based systems have constant false-positive cost per rule and therefore scale poorly when many rules are layered. Cao et al. [2] applied particle-swarm-optimised classifiers to labelled synthetic and exchange data, treating manipulation detection as a per-order or per-trader binary classification.

The graph turn is more recent. The genuinely collusive activity that modern markets observe—ring trading, clique cross-trading, front account schemes—does not have a per-trader signature. Each individual account in a ring may look unremarkable; the conspiracy lives in the edges. Graph neural networks have therefore become a natural fit, and several surveys discuss graph-based financial fraud detection [3]. We use GraphSAGE [4] for its inductive properties: production deployments need to operate on graphs whose node identities differ between training and test ses-

sions. Spectral approaches such as GCN [6] are widely used but less natural in the inductive setting, and attention-based architectures such as GAT [5] could in principle be substituted for the backbone we adopt.

Cross-distribution evaluation has been emphasised in image classification by Recht et al. [9] and made systematic in the WILDS [10] benchmark. The analogue in surveillance is to vary not only the seeds and parameter envelope but also the underlying simulator; we use ABIDES [11] as a structurally different second generator for that test. For the synthesiser’s stylised facts we follow Cont [12].

3 Pipeline Overview

The pipeline comprises five layers shown schematically below. Each layer communicates with the next through an explicit, versioned artifact, which keeps each layer independently testable and re-runnable.

1. Daily NSE bhavcopy fetch and calibration → parameters of stylised-facts synthesiser
2. Synthesiser run (with optional ABIDES substrate) → **orders.csv** and **trades.csv** per scenario
3. Graph construction and feature extraction → per-run PyG graphs and per-trader feature CSVs
4. Detector inference (baseline / bolt-on / augmented) → per-trader probability scores
5. Investigation dashboard (Streamlit + LLM) → investigator-facing flag list with evidence

Each artifact is small, versioned by cohort, and cached. Re-running a single layer is cheap: feature extraction over a 50-run cohort completes in under two minutes on CPU; the bolt-on classifier under leave-one-run-out cross-validation completes in under 45 seconds; inference on a 50-run cohort takes less than a second on GPU. This makes the pipeline iterable enough that ablations and operating-point sweeps are routine rather than expensive.

4 Data Pipeline

4.1 NSE Calibration Loop

The data pipeline begins with daily NSE bhavcopy ingestion. A small fetcher service downloads end-of-day bhavcopy files for the targeted ticker universe and stores them in SQLite. A calibrator service estimates the first and second moments of log-returns over a rolling window and emits a calibration manifest that the synthesiser ingests as its base regime. This manifest is the only direct link between real data and synthetic data; everything downstream is generated, labelled and varied within the controllable envelope.

4.2 Primary Synthesiser

The primary synthesiser produces a daily session of orders and trades for a configurable population (typically 500 traders). It follows the Cont stylised facts: heavy-tailed log-returns, volatility clustering, absence of linear return autocorrelation. Manipulation scenarios are injected on top of an otherwise benign session via three families: *clique* (a small set of accounts pushing or absorbing price through a shared front), *ring* (a closed cycle of accounts passing positions around), and *mixed* (a session combining both patterns). The injection layer is parameterised by core manipulator count, manipulation ratio, duration, and concealment level—giving us the controllable envelope we use to define training, evaluation, and cross-distribution cohorts.

4.3 ABIDES Substrate

A second substrate is the ABIDES agent-based discrete-event simulator [11], used here as a cross-generator stress test rather than a primary training source. We implemented three ABIDES agents (`CollusiveCliqueAgent`, `RingTraderAgent`, `FrontAccountAgent`) that approximate the same three taxonomies used in the primary synthesiser, but whose underlying microstructure (limit order book dynamics, exchange agent behaviour, message ordering) differs substantially. ABIDES sessions are then run through the same downstream pipeline as primary-synthesiser sessions; an adapter writes ABIDES output to the standard `orders.csv/trades.csv/scenarios.csv/traders.csv` schema so that the detector and dashboard layers are generator-agnostic.

4.4 Cohort Definitions

Three cohorts are used in this paper. The training cohort consists of five days at ~ 50 runs per day, drawn from a wide parameter envelope. The within-generator OOD cohort consists of 50 runs from the primary synthesiser, drawn from parameter ranges strictly outside the training envelope (session duration 60–90 min, manipulation ratio 2–5%, core count 16–25) with seeds disjoint from training. The cross-generator cohort consists of 15 ABIDES runs (5 seeds across 3 families on a held-out calibration date), supplementing a 3-run pilot. Table 1 summarises.

5 Feature Engineering for Surveillance

The pipeline includes six per-trader features computed over each trader’s busiest five-minute order window.

Table 1: Cohort summary.

Cohort	Runs	Traders/run	Manip./run
Training (5-day, primary)	250	~ 500	$\sim 10\text{--}20$
Within-generator OOD	50	~ 500	16–25
ABIDES pilot	3	~ 125	6
ABIDES expanded	15	~ 125	6

Each feature was chosen so that an investigator can be told, in one sentence, what manipulation behaviour it captures. This is deliberate: a feature whose meaning a human cannot articulate is not deployable in a surveillance setting, even if it is statistically powerful. Below, O_v denotes the order set of trader v ; B_v is the five-minute window with the maximum order count within O_v ; T_v is the set of trades in which v participated during B_v ; s_k is the share of T_v attributable to counterparty k ; and q is the vector of order quantities placed by v .

1. **Burst concentration**, $\phi_1(v) = |B_v|/|O_v|$. A ring or clique that spikes activity in a tight window produces $\phi_1 \gg 0$; a steadily active legitimate trader produces $\phi_1 \approx 0.1$.
2. **Side entropy in burst**, $\phi_2(v) = -p \log_2 p - (1-p) \log_2 (1-p)$, where p is the buy fraction inside B_v . A pump tends to be one-sided ($\phi_2 \rightarrow 0$); a ring rotating positions tends to be balanced ($\phi_2 \rightarrow 1$).
3. **Counterparty HHI**, $\phi_3(v) = \sum_k s_k^2$. A front-account scheme concentrates burst trades against one or two favoured counterparties; ring members deliberately spread.
4. **Order quantity coefficient of variation**, $\phi_4(v) = \sigma(q)/\mu(q)$. Synthetic manipulators often place uniformly-sized orders; benign traders vary.
5. **Top-partner trade share**, $\phi_5(v) = \max_k s_k$. A direct readout of the strongest single counterparty link.
6. **Co-active top-quantile count**, $\phi_6(v)$: the number of other top-activity-quantile traders whose burst windows overlap that of v . A structural marker for coordinated activity.

The standalone power of each feature on the primary OOD cohort is summarised in Table 2. Importantly, the *direction* of each feature is learned from data rather than hand-coded: on this cohort, the manipulator direction for ϕ_3 is in fact *low* (because the ring scenarios spread counterparties), which is the opposite of the

Table 2: Standalone ROC AUC of engineered features (sign-agnostic).

Feature	Standalone AUC	Direction
ϕ_3 Counterparty HHI in burst	0.800	—
ϕ_5 Top-partner trade share	0.775	—
ϕ_6 Co-active top-quantile count	0.744	—
ϕ_1 Burst concentration	0.704	—
ϕ_4 Order qty coefficient of variation	0.666	+
ϕ_2 Side entropy in burst	0.550	—

textbook intuition. The ensemble downstream therefore needs to be sign-agnostic; this is the specific reason we use a gradient-boosted tree ensemble as the bolt-on classifier rather than a linear model.

6 Detector Stack

The pipeline’s detector layer is deliberately modular. Three variants are evaluated.

6.1 Baseline GraphSAGE (v1)

A two-layer GraphSAGE network with `max` aggregator, hidden dimensions (256, 128, 64), dropout 0.3. The input is the directed trade graph of one session; the output is a per-edge manipulation probability. Node features are two-dimensional—trader’s total executed quantity and number of distinct counterparties—which we treat as a graph-topology baseline. Edge features are eight-dimensional, covering lead-lag correlation, signed imbalance, traded volume, interaction count, pre- and post-window mid-price drift, and VWAP imbalance. The model trains with focal loss ($\alpha=0.85, \gamma=2.0$) and a five-day continual warm-start schedule. Early stopping is governed by per-family validation trader recall rather than validation cross-entropy, in keeping with the surveillance operating metric.

6.2 Tier-2 Bolt-On Classifier

The second variant attaches a gradient-boosted classifier to the baseline’s per-trader score. The classifier consumes a seven-element input vector: the baseline GraphSAGE’s trader score plus the six engineered features of §5. Hyperparameters are $n = 120$ estimators, depth 3, learning rate 0.08, subsample 0.85. The classifier is trained under leave-one-run-out cross-validation over the 50-run OOD cohort, producing pooled per-trader probabilities that we then sweep for an operating point. This bolt-on stack is the simplest, cheapest, and most interpretable configuration; it inherits the baseline GraphSAGE checkpoint without modification and adds 40 seconds of CPU compute.

Table 3: Hyperparameters held constant across all comparisons.

Component	Parameter	Value
GraphSAGE	layers	2
	hidden dims	256, 128, 64
	aggregator	max
	dropout	0.3
	loss	focal ($\alpha=0.85, \gamma=2.0$)
	early-stop patience	6 (val trader recall)
Tier-2 GBM	n_estimators	120
	max_depth	3
	learning_rate	0.08
	subsample	0.85
Both	random seed	42

6.3 Feature-Augmented GraphSAGE (v4)

The third variant feeds the six engineered features into the network itself as additional node attributes. The node-feature vector grows from two dimensions to eight, and the network is retrained from scratch (the first convolution layer’s weight matrix changes shape, precluding warm-start from the baseline checkpoint). All other hyperparameters and the training schedule are held fixed, so the only variable between the baseline and the augmented network is the node-feature set.

6.4 Implementation Details

All graph network training and inference runs in a Docker container provisioned with PyTorch 2.x and PyTorch Geometric, on an NVIDIA RTX 5080 workstation card (sm_120, 16 GB VRAM). The bolt-on classifier uses scikit-learn’s `GradientBoostingClassifier`. Graph construction and feature extraction are pure NumPy/pandas and are CPU-portable. Hyperparameters held constant across all comparisons are listed in Table 3. A single five-day continual training run for the augmented network takes approximately 170 minutes on the workstation card; feature caching for the OOD cohort completes in under two minutes on CPU; the bolt-on classifier under leave-one-run-out cross-validation (50 folds, $\sim 10,000$ trader-rows) completes in under 45 seconds on a single CPU core; inference on the 50-run OOD cohort for either network takes well under one second on GPU.

6.5 Pipeline Orchestration

The five pipeline layers communicate through file-based artifacts rather than in-memory handoff, which keeps each layer independently runnable and makes the pipeline easy to resume after a crash. The training loop is summarised below.

Pipeline orchestration (one cohort end-to-end):

1. Fetch NSE bhavcopy for the trading day and persist to SQLite.
2. Calibrate the synthesiser parameters from the latest bhavcopy window; persist a calibration manifest.
3. Generate the cohort using the calibrated synthesiser (or ABIDES); write `orders.csv`, `trades.csv`, `scenarios.csv`, `traders.csv` per run.
4. Extract per-run engineered features and cache to `outputs/_phase_g_features/`.
5. Build per-run graphs (cached to disk after first build).
6. For each detector variant: load matching graphs, train or load checkpoint, score per trader, persist to `outputs/_phase_g_v{N}_trader_scores/`.
7. Sweep operating-point thresholds; persist a per-detector operating-points JSON for the dashboard.
8. Launch the Streamlit dashboard pointing at the cohort directory.

The atomic step in the loop is step 6: every other step caches its output. A single ablation (re-running step 6 with a different detector variant) typically completes in tens of seconds on cached features and graphs.

7 Trader Projection and Operating-Point Selection

The detector layer emits per-edge probabilities; the dashboard and the investigator workload calculus need per-trader scores. We use a weighted projection of each trader’s incident-edge probabilities,

$$\hat{y}(v) = 0.7 \max_{e \in E(v)} p_e + 0.3 \frac{1}{3} \sum_{e \in \text{top}_3(v)} p_e, \quad (1)$$

which balances the strong-signal bias of a pure `max` (which over-weights single noisy edges and hurts purity) against the dilution of a pure mean.

The threshold applied to $\hat{y}$ is then chosen by an investigator-workload calculus rather than by a single statistical criterion. For an OOD cohort whose pooled labels are known, sweeping the threshold produces a recall/purity curve with several natural operating points. We surface three of them as defaults (Table 4): a high-precision point useful when investigator capacity is constrained, a balanced point at the best F_1 , and a high-recall point useful when investigator capacity is loose. The dashboard exposes a slider so that an investigator can move along the curve and watch the per-flag count change in real time.

Table 4: Operating points for the tier-2 classifier on the primary OOD cohort.

Operating point	Threshold	Recall	Purity
High-precision	0.86	0.619	0.991
Best F_1	0.48	0.759	0.941
High-recall	0.18	0.850	0.803

8 Investigation Dashboard

The dashboard is built on Streamlit and consumes only the artifacts emitted by the layers above. Its job is not to retrain or score—all scoring is pre-computed—but to assemble each flagged trader’s evidence in a single view. Six sections appear in sequence:

1. **Run selector and cohort context.** A drop-down of the fifty OOD runs with a one-line summary (family, seed, parameter slice).
2. **Feature library.** The six features with their descriptions, the live sample value for a flagged trader, and the standalone AUC.
3. **Flagged-trader 2D scatter.** Trader score on one axis, a selectable feature on the other; the threshold slider drags a dashed horizontal line and the flagged set updates instantly.
4. **Manipulated-ticker price and volume.** One-minute mid price and volume of the ticker the manipulator operated on, with a shaded band marking the burst window.
5. **Counterparty network.** A two-ring graph showing the top-flagged traders inside and their counterparties outside, with edge thickness proportional to trade count.
6. **LLM justification.** For each flagged trader, a short sentence generated by a local model (qwen2.5:7b) constrained by the feature values, the per-trader trade count, and the ground-truth label where available. The model paraphrases the numbers into prose; it does not introduce evidence that the numbers do not support.

In practice the dashboard is consulted both at training time (to identify which features are doing what work, often by visual inspection of the scatter) and at deployment time (to give an investigator a single page per flagged trader with all relevant evidence laid out).

8.1 Dashboard Layout

Figure 1 sketches the dashboard layout. The flow is top-to-bottom: cohort selection $\rightarrow$ feature library $\rightarrow$ flagged-trader scatter $\rightarrow$ price/volume context $\rightarrow$

Streamlit dashboard:

- A. Run selector + cohort metrics
- B. Feature library (6 features, sample value, AUC)
- C. Flagged-trader 2D scatter + threshold slider
- D. Manipulated-ticker price/volume with burst-window band
- E. Counterparty network (focal traders + counterparties)
- F. Per-trader LLM justification cards

Figure 1: Dashboard layout (per-cohort view). Each section consumes only pre-computed artifacts; no model inference happens inside Streamlit.

counterparty network $\rightarrow$ per-trader LLM justification. Each chart is accompanied by a small explanatory box, generated once per cohort and cached for one hour, that paraphrases the chart’s specific numbers in plain English. This is a deliberate design choice: investigators tend to skim charts, and a one-sentence plain-language summary substantially improves the signal that the chart’s contents are absorbed before the investigator moves on.

8.2 LLM Justification Layer

The LLM justification card is generated by a local model (qwen2.5:7b via Ollama) prompted with the flagged trader’s six feature values, total order count, in-burst order count, and the ground-truth label where available. The prompt is structured so that the model paraphrases the supplied facts into a two-sentence verdict; it is not given a free hand to introduce evidence the numbers do not support. Cards are cached for one hour, keyed on the prompt string, so the dashboard pays the model cost once per cohort even if multiple investigators consult it. The card is intended as a *speech-act layer*: it converts numbers into prose, lowering the cognitive cost for an investigator who needs to triage many flags quickly.

9 Empirical Validation

We validate the pipeline against three increasingly demanding cohorts.

9.1 Within-Generator OOD

Table 5 summarises the primary result. The baseline GraphSAGE achieves edge AUC 0.638, per-family trader recall 0.41–0.48, and per-family purity 0.09–0.14—a clearly inadequate operating point. The augmented network achieves edge AUC 0.984, per-family recall 0.74–0.93, and purity 0.71–0.77. The bolt-on

Table 5: Within-generator OOD results ($n=50$ runs).

Metric	v1 baseline	Tier-2 GBM	v4 augmented
Edge AUC	0.638	—	0.984
Trader AUC (pooled)	0.793	0.968	—
Clique recall	0.41	—	0.81
Ring recall	0.48	—	0.93
Mixed recall	0.41	—	0.75
Recall @ 99% purity	—	0.62	—
Recall @ 95% purity	—	0.74	—

Table 6: Cross-generator (ABIDES expanded, $n=15$) result.

Metric	v1 baseline	v4 augmented
Edge AUC	0.518 (chance)	0.842
Clique recall (mean $\pm$ sd)	0.867 ± 0.16	0.905 ± 0.11
Ring recall	0.864 ± 0.03	1.000 ± 0.00
Mixed recall	0.837 ± 0.07	0.983 ± 0.02
Clique purity (aggregate)	0.096	0.094
Ring purity	0.276	0.293
Mixed purity	0.249	0.269

classifier achieves pooled trader AUC 0.968 and operating points of 62% recall at 99% purity or 74% recall at 95% purity.

9.2 Cross-Generator (ABIDES)

We re-run inference for the baseline and the augmented network on the expanded ABIDES cohort (Table 6). The augmented network retains edge AUC 0.842; the baseline collapses to chance (0.518). Per-family v4 mean recall remains high (ring 1.000 ± 0.000 , mixed 0.983 ± 0.015 , clique 0.905 ± 0.113). Purity, however, does not transfer: both detectors land in the 0.10–0.29 range on ABIDES, much lower than v4’s 0.71–0.77 within-generator. The fact that the baseline and the augmented network land at the *same* ABIDES purity range argues that the purity gap is a cohort-level threshold-calibration issue rather than an architectural weakness of v4. We return to this in §10.

9.3 Leave-One-Family-Out

The most diagnostic test for the pipeline is to hold out an entire manipulation family from training and ask the detectors to recognise it cold. For the bolt-on stack this is a leave-one-family-out cross-validation over the trader-level data. For the augmented network it requires three full retrainings, each with one family’s runs removed from the cohort and the per-day validation pool. Table 7 reports both.

The bolt-on classifier holds AUC 0.91–0.99 on the held-out family. The augmented network collapses to chance or below. We discuss this contrast in detail in §10; the short version is that the bolt-on stack consumes trader-marginal features, while the augmented network’s message-passing layers learn family-specific

Table 7: Leave-one-family-out AUC.

Held-out family	Tier-2 GBM AUC	v4 retrained AUC
Ring	0.994	0.480
Clique	0.975	0.533
Mixed	0.906	0.374

Table 8: Leave-one-feature-out ablation through the bolt-on classifier.

Feature dropped	Δ AUC	Recall @ p99
full (7 features)	—	0.619
ϕ_1 burst concentration	−0.051	0.274
ϕ_6 co-active count	−0.007	0.592
ϕ_2 side entropy	−0.004	0.634
ϕ_4 order qty CoV	−0.003	0.608
ϕ_3 counterparty HHI	−0.002	0.620
ϕ_5 top-partner share	+0.000	0.625
baseline v1 score	+0.000	0.530

neighbourhood signatures that do not transfer. A methodological caveat applies to the v4 result: excluding a family from training shrinks the per-day cohort by a third, which in turn shrinks the validation pool to three graphs and forces an aggressive early-stop trajectory. A longer training schedule with a larger validation pool would likely recover some of the lost AUC, and this remains the most important follow-up experiment.

9.4 Feature Ablation Inside the Bolt-On

A leave-one-feature-out ablation through the bolt-on classifier (Table 8) reveals an uneven distribution of load. Dropping ϕ_1 (burst concentration)—only fifth-ranked by standalone AUC—costs −0.051 AUC and collapses recall at 99% purity from 0.62 to 0.27. Dropping ϕ_2 (side entropy), the weakest feature standalone, costs only −0.004 AUC. The standalone ranking is therefore a poor proxy for ensemble importance: a feature that is weak in isolation can be load-bearing in combination, and a feature that is strong in isolation can be substitutable.

9.5 Case Study: One Ring Run From OOD

We pick a representative ring run from the primary OOD cohort and trace the pipeline’s behaviour. The run contains ~ 200 traders, six of whom are members of a ring rotating positions among themselves over a single ~ 45 -minute window. The six manipulator traders share three salient properties: burst concentration $\phi_1 > 0.85$ (all six, versus a benign median of 0.16); low counterparty HHI $\phi_3 < 0.2$, reflecting the ring’s intentionally diversified counterparty list; and

high co-active count ϕ_6 because the six rotate trades among themselves and against each other simultaneously. The dashboard surfaces all six on the flagged-trader scatter at any threshold above 0.55; the counterparty network displays a closed cycle of six red focal nodes connected by thick edges; the per-trader LLM cards each contain the phrase “ring signature” or “cycle structure”. The investigator’s verification is then a question of (i) noting the cycle in the counterparty network, (ii) confirming the burst-window shading on the price chart aligns with the LLM card, and (iii) clicking through to the underlying trades. End-to-end, this is the workflow the pipeline was designed for.

9.6 Per-Run Variance on the ABIDES Cohort

On the expanded ABIDES cohort, the per-run breakdown deserves attention because three of the fifteen runs are responsible for most of the spread in clique recall. Across the five ring runs the augmented network’s recall is exactly 1.000 with zero variance; across the five mixed runs it is 0.98 ± 0.02 ; across the five clique runs it is 0.91 ± 0.11 with one outlier at 0.69. The outlier corresponds to a clique scenario whose front counterparties happened to receive unusually balanced order flow, partially masking the burst-concentration signature. This bimodality is a fair argument for surfacing per-run distributions in the dashboard alongside the cohort-mean numbers.

10 Deployment Considerations

The empirical results imply a clear deployment narrative. *Ranking* generalises across distributions: the augmented network holds AUC 0.842 on ABIDES versus the baseline’s chance, and the bolt-on stack holds AUC 0.91–0.99 leave-one-family-out. *Operating points* do not generalise: purity collapses in the cross-generator regime, and the locked threshold tuned on one cohort flags too many benigns on a structurally different one. The right production hygiene is therefore per-generator (or per-deployment) threshold calibration, not per-generator retraining.

The pipeline supports this directly. Threshold calibration is the cheapest layer to re-run—no model retraining, no graph rebuild, just a threshold sweep over pre-computed per-trader scores. We argue that this should be part of any production rollout plan: deploy the detector with a default threshold drawn from the training cohort, hold out a small calibration cohort from the operational distribution, and sweep the threshold on the calibration cohort to find the operating point that matches the surveillance team’s purity floor.

The architectural contrast between the bolt-on and the end-to-end detector also has a practical consequence. For deployments where the operational distribution is expected to drift across structural axes (different exchanges, different ticker classes, different manipulation taxonomies than were anticipated at training), the bolt-on stack is the more robust choice. For deployments where the operational distribution is well-characterised at training and the goal is to maximise within-distribution recall at high purity, the augmented network wins comfortably. The pipeline supports both, and the choice is a deployment decision rather than a methodological one.

11 Discussion: Why Bolt-On Beats End-to-End on Family Shift

The most provocative empirical result of this work is the leave-one-family-out contrast: the bolt-on classifier retains AUC 0.91–0.99 on a held-out family, while the augmented network collapses to chance. We argue this contrast is informative for system design and is not just a methodological accident.

Trader-marginal vs. relational scoring. The bolt-on classifier consumes a seven-dimensional per-trader vector and decides each trader’s score independently of the others. Its decision surface therefore depends only on whether the engineered features describe a manipulator-like trader *in isolation*. Because the features were chosen to be taxonomy-grounded but family-agnostic (ϕ_1 – ϕ_6 describe what manipulation looks like, not what clique manipulation looks like), the decision surface transfers cleanly across families.

Family-specific neighbourhood priors. The augmented network, in contrast, performs two layers of message passing before emitting an edge probability. The message-passing step pools each node’s neighbour feature vectors, which means the network can—and empirically does—learn that “a node whose neighbourhood resembles a ring” or “a node whose neighbourhood resembles a clique” is suspicious. These neighbourhood signatures are family-specific by construction. When a family is held out from training, the network’s structural prior for that family is absent, and the trader-marginal signal it does see (via the same six engineered features) is over-ridden by the inductive bias of the message-passing layers.

Implication for system design. For deployments where the operational distribution is expected to drift

across structural axes (a new exchange whose manipulators differ taxonomically from the training distribution; a regulator-imposed microstructure change that shifts neighbourhood patterns), the bolt-on stack is the more robust choice. For deployments where the operational distribution is well-characterised at training and the goal is to maximise within-distribution recall at high purity, the augmented network wins comfortably. The pipeline supports both, and we view the architectural choice as a deployment-time decision rather than a model-evaluation decision.

When the contrast might not hold. Two caveats are important. First, the v4 family-disjoint result was obtained under the five-day training schedule that was tuned for the full cohort, and the schedule’s interaction with a reduced cohort is partially responsible for the v4 collapse. With a longer training schedule and a larger validation pool (e.g. 20% of per-day runs rather than 10%, which the reduced cohort already constrains to about three graphs), we expect v4’s family-disjoint performance to recover at least partially. This is a clear follow-up experiment. Second, the message-passing inductive bias that hurts the v4 family-disjoint result is the *same* bias that lifts v4 to AUC 0.984 within-distribution and 0.842 on the cross-generator cohort; the two are properties of the same architecture, not separable knobs.

Operational workflow. A pragmatic deployment that wants both robustness and within-distribution lift would run the bolt-on classifier as the primary detector for flag generation, and use the augmented network as a confirmation layer in the dashboard. Flags where both detectors agree could be promoted to top-of-queue; disagreements could be triaged manually or downweighted. The pipeline supports this directly since both detectors emit per-trader scores into the same artifact directory and the dashboard already reads from there.

12 Limitations and Future Work

The most important limitation of the work is that both substrates are synthetic. Although the primary synthesiser is calibrated to NSE bhavcopy first moments and ABIDES adds genuinely different microstructure, neither substrate captures the full heavy-tailed event structure of a real market under stress. A labelled real-data pilot, in which the detector is applied to a slice of historical NSE data with regulator-confirmed manipulation cases, would be the strongest possible validation; the absence of such a labelled cohort is a

recurring limitation in the literature, and we do not solve it here.

Three further limitations apply. First, the family-disjoint v4 retrains used the training schedule tuned for the full cohort, and the schedule’s interaction with a reduced training set and reduced validation pool is responsible for at least some of the observed collapse. A separate study with a longer schedule and larger validation pool is indicated. Second, the LLM justification card is not formally validated for factual consistency with the underlying feature values; we constrain the prompt to the supplied feature context, but a separate evaluation of the justification card’s faithfulness is needed. Third, the operating-point selection methodology assumes the investigator team has a single purity floor, which may be an oversimplification of the per-flag triage workflow.

13 Threats to Validity

Four threats to the generalisability of the empirical results deserve explicit listing. First, the locked decision threshold used in the within-generator and ABIDES tables is tuned in-sample on the evaluation cohort, which inflates absolute purity numbers in both detectors equally; the cross-detector comparison is fair, but the absolute operating point is mildly optimistic. Second, both substrates are synthetic. ABIDES adds genuinely different microstructure but is still a simulator. Real-market manipulators may produce patterns that the engineered features were not designed for. Third, the family-disjoint v4 retraining used the training schedule tuned for the full five-day cohort, and the schedule’s interaction with a reduced cohort and reduced validation pool is responsible for at least some of the observed v4 collapse. A retraining with looser early-stopping and a larger validation pool would be a more definitive test of the architectural contrast. Fourth, the LLM justification card has been constrained by prompt structure rather than validated for factual consistency with the underlying feature values; an independent faithfulness evaluation is required before the card is treated as audit evidence.

14 Reproducibility

All artifacts—calibration manifests, synthetic cohorts, feature caches, model checkpoints, dashboard configurations—are persisted as plain files in a single `outputs/` directory, versioned by cohort name and detector variant. The training pipeline runs entirely inside Docker containers; a single command (`docker compose run`) launches the trainer container against the workstation GPU. The bolt-on classifier runs on CPU outside Docker as a standalone Python script

and completes in under a minute. The dashboard is one Streamlit process pointed at the same `outputs/` directory.

15 Conclusion

We have presented a surveillance pipeline for trade-market manipulation detection that ties together five components: a daily NSE calibration loop, a calibrated synthetic substrate, a second substrate (ABIDES) used for cross-generator stress testing, a modular detector layer comprising a graph neural network with optional second-stage classifier, and an investigator dashboard that surfaces flagged traders with feature-level evidence and a language-model justification card. The pipeline lifts the detector’s within-generator out-of-distribution edge AUC from 0.638 to 0.984, achieves pooled trader AUC 0.968 in the bolt-on configuration, and retains edge AUC 0.842 on a cross-generator cohort drawn from a different simulator. A leave-one-family-out evaluation reveals an unexpected architectural contrast: the bolt-on stack maintains AUC 0.91–0.99 on the held-out family while the augmented network collapses to chance, suggesting that for deployments where structural distribution shift is expected, the simpler classifier is a more robust choice. The pipeline supports per-generator threshold calibration as the recommended deployment hygiene for the cross-distribution operating-point issue. We argue the pipeline is the contribution: each layer was motivated by a specific surveillance requirement, and the interfaces between layers are simple enough that individual components can be swapped without disturbing the rest.

References

- [1] S. Khanna and A. Aggarwal, “A survey of market manipulation detection: trends and open problems,” *Journal of Financial Surveillance*, vol. 8, no. 3, pp. 145–168, 2021.
- [2] Y. Cao, Y. Li, S. Coleman, A. Belatreche, and T. McGinnity, “Particle-swarm-optimised support-vector machine for detecting market manipulation,” in *Proc. IEEE Symp. on Computational Intelligence for Financial Engineering*, 2014.
- [3] T. Sun, K. Liu, X. Chen, and H. Ye, “Graph neural networks for financial-fraud detection: a survey,” *ACM Computing Surveys*, 2022.
- [4] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive representation learning on large graphs,”

- in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [5] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *Int. Conf. on Learning Representations (ICLR)*, 2018.
 - [6] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *Int. Conf. on Learning Representations (ICLR)*, 2017.
 - [7] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, “Focal loss for dense object detection,” in *Proc. IEEE Int. Conf. on Computer Vision (ICCV)*, 2017.
 - [8] J. H. Friedman, “Greedy function approximation: a gradient boosting machine,” *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.
 - [9] B. Recht, R. Roelofs, L. Schmidt, and V. Shankar, “Do ImageNet classifiers generalize to ImageNet?” in *Proc. Int. Conf. on Machine Learning (ICML)*, 2019.
 - [10] P. W. Koh et al., “WILDS: A benchmark of in-the-wild distribution shifts,” in *Proc. Int. Conf. on Machine Learning (ICML)*, 2021.
 - [11] D. Byrd, M. Hybinette, and T. H. Balch, “ABIDES: Towards high-fidelity multi-agent market simulation,” in *Proc. ACM SIGSIM Conf. on Principles of Advanced Discrete Simulation*, 2019.
 - [12] R. Cont, “Empirical properties of asset returns: stylized facts and statistical implications,” *Quantitative Finance*, vol. 1, no. 2, pp. 223–236, 2001.
 - [13] T. Chen and C. Guestrin, “XGBoost: a scalable tree boosting system,” in *Proc. ACM SIGKDD Conf. on Knowledge Discovery and Data Mining*, 2016.
 - [14] Y. Dou, Z. Liu, L. Sun, Y. Deng, H. Peng, and P. S. Yu, “Enhancing graph neural network-based fraud detectors against camouflaged fraudsters,” in *Proc. ACM Int. Conf. on Information and Knowledge Management (CIKM)*, 2020.
 - [15] W. Cong, H. K. Suarez, L. Akoglu, and M. Mahdavi, “AMLSim: a synthetic data generator for anti-money-laundering research,” *IEEE BigData Workshop on Financial Fraud*, 2020.
 - [16] Streamlit Team, “Streamlit: the fastest way to build and share data apps,” <https://streamlit.io/>, accessed 2026.
 - [17] H. Nguyen, P. Phung, and Y. Wang, “Towards out-of-distribution generalisation in graph neural networks: a benchmark,” in *Proc. Int. Conf. on Learning Representations (ICLR) Workshop on ML for Finance*, 2023.
 - [18] J. Leskovec and R. Sosič, “SNAP: a general-purpose network analysis and graph-mining library,” *ACM Trans. Intell. Syst. Technol.*, vol. 8, no. 1, pp. 1–20, 2016.
 - [19] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, “How powerful are graph neural networks?” in *Int. Conf. on Learning Representations (ICLR)*, 2019.
 - [20] Y. Liu, X. Ao, Z. Qin, J. Chi, J. Feng, H. Yang, and Q. He, “Pick and choose: a GNN-based imbalanced learning approach for fraud detection,” in *Proc. The World Wide Web Conference (WWW)*, 2021.
 - [21] Ollama Team, “Ollama: get up and running with large language models locally,” <https://ollama.com/>, accessed 2026.
 - [22] Docker Inc., “Docker engine reference,” <https://docs.docker.com/engine/>, accessed 2026.